

Systemy dedykowane w układach programowalnych

Euclidean Algorithm (GCD)

Jakub Szwarga, Zuzanna Schab

19 czerwca 2026

1 Wstęp Teoretyczny i Analiza Matematyczna

Algorytm Euklidesa służy do wyznaczania Największego Wspólnego Dzielnika (NWD) dwóch nieujemnych liczb całkowitych a i b . Jego matematyczna prostota wynika z faktu, że w każdej kolejnej operacji wartość NWD tych liczb się nie zmienia:

$$NWD(a, b) = NWD(b, a \bmod b) \quad (1)$$

dla $b \neq 0$.

1.1 Warianty Implementacyjne

1. Wariant przez odejmowanie (klasyczny): wolniejszy w implementacji programistycznej, ale bardzo prosty w strukturach sprzętowych (wymaga jedynie komparatora i układu odejmującego).
2. Wariant przez modulo (zoptymalizowany): znacznie szybszy zbieżnie, redukuje liczbę kroków, jednak w sprzęcie (FPGA/ASIC) wymaga bloku dzielącego, który jest kosztowny zasobowo i generuje duże opóźnienia.

1.2 Analiza Złożoności i Najgorszy Przypadek

Zgodnie z twierdzeniem Lamé, liczba kroków potrzebnych do zakończenia algorytmu dla dwóch liczb nie przekracza pięciokrotności liczby cyfr mniejszej z nich w zapisie dziesiętnym. Najgorszym możliwym przypadkiem dla algorytmu Euklidesa są dwie kolejne liczby z ciągu Fibonacciego (F_n oraz F_{n+1}). Wynika to z faktu, że reszty z dzielenia kolejnych wyrazów tego ciągu generują najmniejsze możliwe zmniejszenie wartości w każdym kroku (ponieważ iloraz wynosi zawsze w przybliżeniu 1).

1.3 Zastosowania Praktyczne i Przemysłowe

1. Kryptografia (algorytm RSA): w kryptografii asymetrycznej klucz publiczny e musi być względnie pierwszy z wartością funkcji Eulera $\phi(n)$. System sprawdza ten warunek, weryfikując czy $NWD(e, \phi(n)) = 1$. Rozszerzony Algorytm Euklidesa (EEA) jest następnie używany do wyznaczenia klucza prywatnego d , będącego odwrotnością modularną e .
2. Przetwarzanie sygnałów i systemy wbudowane: w architekturach FPGA, moduł wyznaczający NWD wykorzystuje się m.in. w:

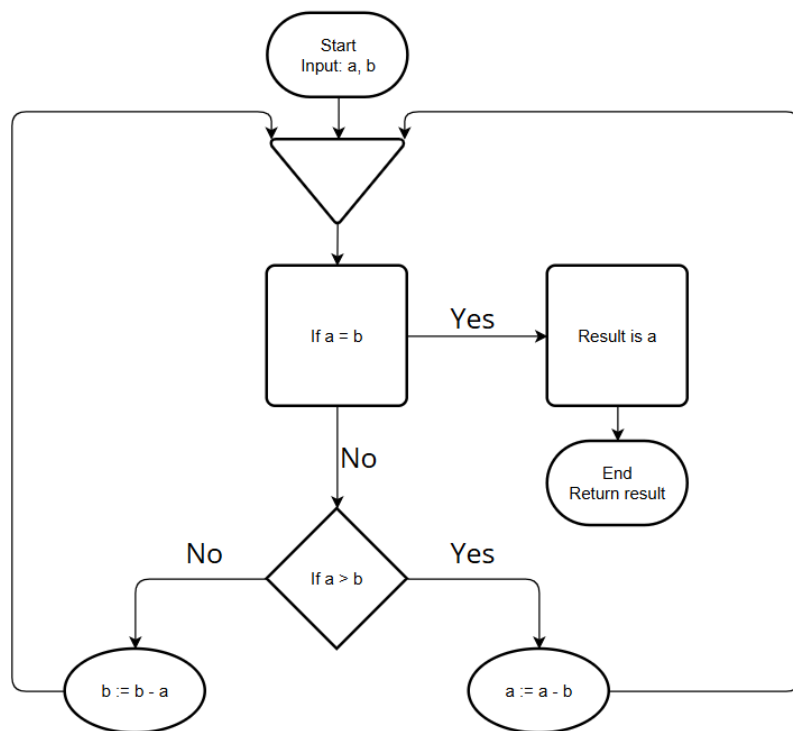
- synchronizacji zegarów o różnych częstotliwościach (generowanie sygnałów bazowych),
- algorytmach skalowania obrazu (wyznaczanie proporcji aspektu - Aspect Ratio),
- redukcji danych w strumieniach sieciowych (dopasowywanie pakietów).

2 Wprowadzenie

Projekt realizuje akcelerator sprzętowy NWD (Największy Wspólny Dzielnik) zaimplementowany na platformie ZedBoard. System integruje część procesorową (PS) z logiką programowalną (PL) poprzez interfejs AXI4-Lite, oferując równoległe przetwarzanie czterech kanałów obliczeniowych.

2.1 Schemat blokowy działania skryptu

Poniższy diagram przedstawia logiczny przebieg testów porównawczych zaimplementowanych w języku Python.



Rysunek 1: Schemat działania algorytmu testowego

3 Analiza Algorytmów NWD

W celu optymalizacji akceleratora porównano algorytm modulo oraz metodę odejmowania. Wersja oparta na operacji modulo wykazuje znaczną przewagę wydajnościową, dlatego została wybrana do implementacji sprzętowej.

Listing 1: Porównanie algorytmów w Pythonie

```

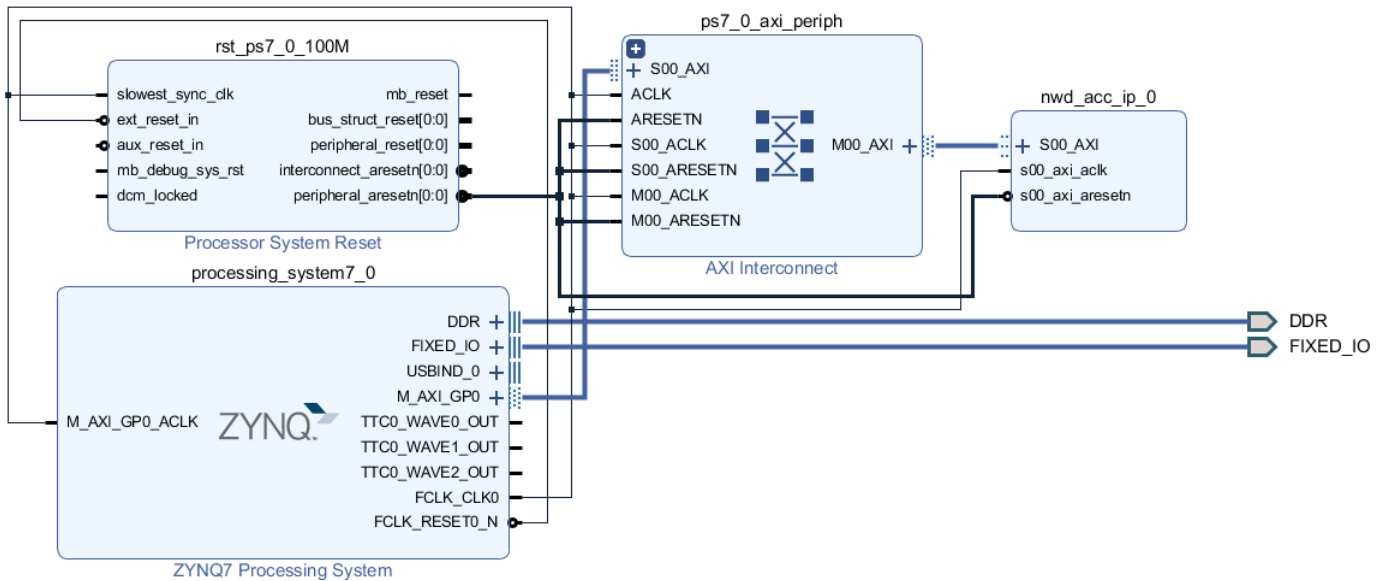
1 def euklides_modulo(a: int, b: int) -> tuple:
2     # Implementacja algorytmu Euklidesa modulo
3     kroki = 0
4     while b != 0:
5         a, b = b, a % b
6         kroki += 1
7     return a, kroki
8
9 def euklides_odejmowanie(a: int, b: int) -> tuple:
10    # Implementacja algorytmu Euklidesa odejmowanie
11    kroki = 0
12    if a == 0: return b, kroki
13    if b == 0: return a, kroki
14
15    while a != b:
16        if a > b:
17            a = a - b
18        else:
19            b = b - a
20        kroki += 1
21    return a, kroki
22
23 def uruchom_testy():
24     przyklady = [
25         (105, 24),
26         (1071, 462),
27         (55, 34),          # Liczby Fibonacciego
28         (1000, 2)          # Skrajny przypadek
29     ]
30
31     print("POROWNANIE ALGORYTMOW EUKLIDESA")
32     print("=" * 60)
33
34     for a, b in przyklady:
35         wynik_mod, kroki_mod = euklides_modulo(a, b)
36         wynik_odej, kroki_odej = euklides_odejmowanie(a, b)
37
38         print(f"Dane wejsciowe: a = {a}, b = {b}")
39         print(f"  -> NWD (Modulo):          {wynik_mod}, Kroki: {kroki_mod}")
40         print(f"  -> NWD (Odejmowanie):      {wynik_odej}, Kroki: {kroki_odej}")
41         print("-" * 60)
42
43 if __name__ == "__main__":
44     uruchom_testy()

```

4 Diagram Blokowy Systemu

System opiera się na wydajnej współpracy między procesorem, który zarządza logiką programu, a logiką programowalną, w której zaimplementowano akcelerator sprzętowy. Komunikacja od-

bywa się za pośrednictwem interkonektu AXI, który mapuje przestrzeń adresową akceleratora tak, aby procesor mógł zapisywać dane wejściowe i odczytywać wyniki NWD w sposób zsynchronizowany.



Rysunek 2: Schemat blokowy połączeń IP Integrator

5 Implementacja Sprzętowa (Verilog)

5.1 Wrapper Projektu (nwd_acc_design_wrapper.v)

```

1 module nwd_acc_design_wrapper
2     (DDR_addr, DDR_ba, DDR_cas_n, DDR_ck_n, DDR_ck_p, DDR_cke, DDR_cs_n,
3       DDR_dm,
4       DDR_dq, DDR_dqs_n, DDR_dqs_p, DDR_odt, DDR_ras_n, DDR_reset_n,
5       DDR_we_n,
6       FIXED_IO_dds_vrn, FIXED_IO_dds_vrp, FIXED_IO_mio, FIXED_IO_ps_clk,
7       FIXED_IO_ps_por_b, FIXED_IO_ps_srstb);
8     inout [14:0] DDR_addr;
9     inout [2:0] DDR_ba;
10    ...
11    nwd_acc_design nwd_acc_design_i
12        (.DDR_addr(DDR_addr), .DDR_ba(DDR_ba), .DDR_cas_n(DDR_cas_n),
13          .DDR_ck_n(DDR_ck_n), .DDR_ck_p(DDR_ck_p), .DDR_cke(DDR_cke),
14          ...

```

5.2 Moduł Główny (nwd_acc_design.v)

```

1 module nwd_acc_design (...);
2     nwd_acc_design_nwd_acc_ip_0_0 nwd_acc_ip_0
3         (.s00_axi_aclk(ps7_0_axi_periph_M00_AXI_ARADDR[5:0]),
4           .s00_axi_araddr(ps7_0_axi_periph_M00_AXI_ARADDR[5:0]),
5           .s00_axi_aresetn(rst_ps7_0_100M_peripheral_aresetn),
6           ...
7 endmodule

```

6 Oprogramowanie Sterujące (main.c)

```
1  ...
2  short load_data() {
3      char c;
4      char buffer[10];
5      int i = 0;
6
7      while(1) {
8          c = inbyte();
9
10         if(c == '\r' || c == '\n') {
11             buffer[i] = '\0';
12             outbyte('\r');
13             outbyte('\n');
14             break;
15         }
16         else if((c == 127 || c == 8) && i > 0) {
17             i--;
18             outbyte('\b');
19             outbyte(' ');
20             outbyte('\b');
21         }
22         else if(i < 9 && ((c >= '0' && c <= '9') || c == '-')) {
23             buffer[i++] = c;
24             outbyte(c);
25         }
26     }
27     return (short)atoi(buffer);
28 }
29
30 int main()
31 {
32     init_platform();
33
34     uint32_t t_input0[4] = {0};
35     uint32_t t_input1[4] = {0};
36     uint32_t t_output[4] = {0};
37
38     while(1) {
39         xil_printf("\n\r=== Interaktywny Test Akceleratora NWD ===\n\r"
40             );
41
42         for(int i = 0; i < 4; i++) {
43             xil_printf("Kanal %d - Podaj pierwsza liczbe: ", i);
44             t_input0[i] = (uint32_t)load_data();
45
46             xil_printf("Kanal %d - Podaj druga liczbe: ", i);
47             t_input1[i] = (uint32_t)load_data();
48
49             xil_printf("-----\n\r");
50         }
```

```

51     xil_printf("Wpisywanie danych do rejestrow AXI...\n\r");
52     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT0_CH0, t_input0[0]);
53     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT0_CH1, t_input0[1]);
54     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT0_CH2, t_input0[2]);
55     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT0_CH3, t_input0[3]);
56
57     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT1_CH0, t_input1[0]);
58     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT1_CH1, t_input1[1]);
59     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT1_CH2, t_input1[2]);
60     Xil_Out32(NWD_ACC_BASEADDR + REG_INPUT1_CH3, t_input1[3]);
61
62     xil_printf("Uruchamianie obliczen sprzetowych...\n\r");
63
64     Xil_Out32(NWD_ACC_BASEADDR + REG_CTRL_STATUS, CTRL_START_MASK);
65
66     uint32_t status = 0;
67     do {
68         status = Xil_In32(NWD_ACC_BASEADDR + REG_CTRL_STATUS);
69     } while ((status & STATUS_READY_MASK) == 0);
70
71     xil_printf("Obliczenia zakonczone! Wyniki:\n\r");
72     t_output[0] = Xil_In32(NWD_ACC_BASEADDR + REG_OUTPUT_CH0);
73     t_output[1] = Xil_In32(NWD_ACC_BASEADDR + REG_OUTPUT_CH1);
74     t_output[2] = Xil_In32(NWD_ACC_BASEADDR + REG_OUTPUT_CH2);
75     t_output[3] = Xil_In32(NWD_ACC_BASEADDR + REG_OUTPUT_CH3);
76
77     Xil_Out32(NWD_ACC_BASEADDR + REG_CTRL_STATUS, 0x00);
78
79     xil_printf("\n\r===== WYNIKI =====\n\r");
80     for(int i = 0; i < 4; i++) {
81         xil_printf("Jednostka %d: NWD(%d, %d) = %d\n\r", i,
82             t_input0[i], t_input1[i], t_output[i]);
83     }
84     xil_printf("=====\n\r");
85
86     xil_printf("\n\rNacisnij dowolny klawisz, aby uruchomic kolejny
87         test...\n\r");
88     inbyte();
89
90     cleanup_platform();
91     return 0;
92 }

```

7 Specyfikacja Techniczna

- **Platforma:** ZedBoard (Zynq-7000).
- **Interfejs:** AXI4-Lite (4 kanały).
- **Tryb pracy:** Równoległe obliczenia sprzętowe (SIMD).